

Concepts

Reports

This guide explains the portfolio analysis and reporting capabilities provided by the `ReportProvider` class, and how these reports are used for PnL accounting and backtest post-run analysis.

Overview

The `ReportProvider` class in NautilusTrader generates structured analytical reports from trading data, transforming raw orders, fills, positions, and account states into pandas DataFrames for analysis and visualization. These reports help you evaluate strategy performance, analyze execution quality, and verify PnL accounting.

Reports can be generated using two approaches:

- **Trader helper methods** (recommended): Convenient methods like `trader.generate_orders_report()`.
- **ReportProvider directly**: For more control over data selection and filtering.

Reports provide consistent analytics across both backtesting and live trading environments, enabling reliable performance evaluation and strategy comparison.

Available reports

The `ReportProvider` class offers several static methods to generate reports from trading data. Each report returns a pandas DataFrame with specific columns and indexing for easy analysis.

Orders report

Generates a full view of all orders:

```
# Using Trader helper method (recommended)
orders_report = trader.generate_orders_report()

# Or using ReportProvider directly
from nautilus_trader.analysis import ReportProvider

orders = cache.orders()
orders_report = ReportProvider.generate_orders_report(orders)
```



Returns `pd.DataFrame`. Key columns include:

Column	Description
<code>client_order_id</code>	Index - unique order identifier.
<code>instrument_id</code>	Trading instrument.
<code>strategy_id</code>	Strategy that created the order.
<code>trader_id</code>	Trader identifier.
<code>account_id</code>	Account identifier (if assigned).
<code>venue_order_id</code>	Venue-assigned order ID (if accepted).
<code>side</code>	BUY or SELL.
<code>type</code>	MARKET, LIMIT, etc.
<code>status</code>	Current order status.
<code>quantity</code>	Original order quantity (string).
<code>filled_qty</code>	Amount filled (string).
<code>price</code>	Limit price (order-type dependent).
<code>avg_px</code>	Average fill price (if filled).
<code>time_in_force</code>	Time-in-force instruction.
<code>ts_init</code>	Order initialization timestamp (Unix nanoseconds).

Column	Description
<code>ts_last</code>	Last update timestamp (Unix nanoseconds).

Additional columns vary by order type (e.g., `trigger_price` for stop orders, `expire_time` for GTD orders). See `Order.to_dict()` for the complete field list.

Order fills report

Provides a summary of filled orders (one row per order):

```
# Using Trader helper method (recommended)
fills_report = trader.generate_order_fills_report()

# Or using ReportProvider directly
orders = cache.orders()
fills_report = ReportProvider.generate_order_fills_report(orders)
```



This report includes only orders with `filled_qty > 0` and contains the same columns as the orders report, but filtered to executed orders only. Note that `ts_init` and `ts_last` are converted to datetime objects in this report for easier analysis.

Fills report

Details individual fill events (one row per fill):

```
# Using Trader helper method (recommended)
fills_report = trader.generate_fills_report()

# Or using ReportProvider directly
orders = cache.orders()
fills_report = ReportProvider.generate_fills_report(orders)
```



Returns `pd.DataFrame`. Key columns include:

Column	Description
<code>client_order_id</code>	Index - order identifier.
<code>trade_id</code>	Unique trade/fill identifier.

Column	Description
<code>venue_order_id</code>	Venue-assigned order ID.
<code>instrument_id</code>	Trading instrument.
<code>strategy_id</code>	Strategy that created the order.
<code>account_id</code>	Account identifier.
<code>position_id</code>	Associated position ID (if applicable).
<code>order_side</code>	BUY or SELL.
<code>order_type</code>	Order type (MARKET, LIMIT, etc.).
<code>last_px</code>	Fill execution price (string).
<code>last_qty</code>	Fill execution quantity (string).
<code>currency</code>	Currency of the fill.
<code>liquidity_side</code>	MAKER or TAKER.
<code>commission</code>	Commission amount and currency.
<code>ts_event</code>	Fill timestamp (datetime).
<code>ts_init</code>	Initialization timestamp (datetime).

See `OrderFilled.to_dict()` for the complete field list.

Positions report

Position analysis including snapshots:

```
# Using Trader helper method (recommended)
# Automatically includes snapshots for NETTING OMS
positions_report = trader.generate_positions_report()

# Or using ReportProvider directly
positions = cache.positions()
snapshots = cache.position_snapshots() # For NETTING OMS
positions_report = ReportProvider.generate_positions_report(
    positions=positions,
```



```
snapshots=snapshots
```

```
)
```

Returns `pd.DataFrame`. Key columns include:

Column	Description
<code>position_id</code>	Index - unique position identifier.
<code>instrument_id</code>	Trading instrument.
<code>strategy_id</code>	Strategy that managed the position.
<code>trader_id</code>	Trader identifier.
<code>account_id</code>	Account identifier.
<code>opening_order_id</code>	Order ID that opened the position.
<code>closing_order_id</code>	Order ID that closed the position.
<code>entry</code>	Entry side (BUY or SELL).
<code>side</code>	Position side (LONG, SHORT, or FLAT).
<code>quantity</code>	Current position size.
<code>peak_qty</code>	Maximum size reached.
<code>avg_px_open</code>	Average entry price.
<code>avg_px_close</code>	Average exit price (if closed).
<code>commissions</code>	List of commissions paid.
<code>realized_pnl</code>	Realized profit/loss.
<code>realized_return</code>	Return percentage.
<code>ts_init</code>	Position initialization timestamp.
<code>ts_opened</code>	Opening timestamp (datetime).
<code>ts_last</code>	Last update timestamp.

Column	Description
<code>ts_closed</code>	Closing timestamp (datetime or NA).
<code>duration_ns</code>	Position duration in nanoseconds.
<code>is_snapshot</code>	Whether this is a historical snapshot.

Account report

Tracks account balance and margin changes over time:

```
# Using Trader helper method (recommended)
# Requires venue parameter
from nautilus_trader.model.identifiers import Venue
venue = Venue("BINANCE")
account_report = trader.generate_account_report(venue)

# Or using ReportProvider directly
account = cache.account(account_id)
account_report = ReportProvider.generate_account_report(account)
```



Returns `pd.DataFrame`. Columns include:

Column	Description
<code>ts_event</code>	Index - timestamp of account state change.
<code>account_id</code>	Account identifier.
<code>account_type</code>	Type of account (e.g., SPOT, MARGIN).
<code>base_currency</code>	Base currency for the account.
<code>total</code>	Total balance amount (string).
<code>free</code>	Available balance (string).
<code>locked</code>	Balance locked in orders (string).
<code>currency</code>	Currency of the balance.
<code>reported</code>	Whether balance was reported by venue.

Column	Description
margins	Margin information (list, if applicable).
info	Additional venue-specific information.

Each row represents a balance entry; accounts with multiple currencies produce multiple rows per account state event.

PnL accounting considerations

Accurate PnL accounting requires careful consideration of several factors:

Position-based PnL

- **Realized PnL:** Calculated when positions are partially or fully closed.
- **Unrealized PnL:** Marked-to-market using current prices.
- **Commission impact:** Only included when in settlement currency.

⚠ PnL calculations depend on the OMS type. In **NETTING** OMS, position snapshots preserve historical PnL when positions reopen. Always include snapshots in reports for accurate total PnL calculation. In **HEDGING** OMS, snapshots are not used since each position has a unique ID and is never reopened.

Multi-currency accounting

When dealing with multiple currencies:

- Each position tracks PnL in its settlement currency.
- Portfolio aggregation requires currency conversion.
- Commission currencies may differ from settlement currency.

```
# Accessing PnL across positions
for position in positions:
    realized = position.realized_pnl # In settlement currency
    unrealized = position.unrealized_pnl(last_price)

# Handle multi-currency aggregation (illustrative)
# Note: Currency conversion requires user-provided exchange rates
```



```

if position.settlement_currency != base_currency:
    # Apply conversion rate from your data source
    # rate = get_exchange_rate(position.settlement_currency, base_currency)
    # realized_converted = realized.as_double() * rate
    pass

```

Snapshot considerations

For **NETTING** OMS:

```

from nautilus_trader.model.objects import Money

# Include snapshots for complete PnL (per currency)
pnl_by_currency = {}

# Add PnL from current positions
for position in cache.positions(instrument_id=instrument_id):
    if position.realized_pnl:
        currency = position.realized_pnl.currency
        if currency not in pnl_by_currency:
            pnl_by_currency[currency] = 0.0
        pnl_by_currency[currency] += position.realized_pnl.as_double()

# Add PnL from historical snapshots
for snapshot in cache.position_snapshots(instrument_id=instrument_id):
    if snapshot.realized_pnl:
        currency = snapshot.realized_pnl.currency
        if currency not in pnl_by_currency:
            pnl_by_currency[currency] = 0.0
        pnl_by_currency[currency] += snapshot.realized_pnl.as_double()

# Create Money objects for each currency
total_pnls = [Money(amount, currency) for currency, amount in pnl_by_currency.items()]

```

Backtest post-run analysis

After a backtest completes, analysis is available through various reports and the portfolio analyzer.

Accessing backtest results

```

# After backtest run
engine.run(start=start_time, end=end_time)

# Generate reports using Trader helper methods
orders_report = engine.trader.generate_orders_report()

```



```
positions_report = engine.trader.generate_positions_report()
fills_report = engine.trader.generate_fills_report()

# Or access data directly for custom analysis
orders = engine.cache.orders()
positions = engine.cache.positions()
snapshots = engine.cache.position_snapshots()
```

Portfolio statistics

The portfolio analyzer provides performance metrics:

```
# Access portfolio analyzer
portfolio = engine.portfolio

# Get different categories of statistics
stats_pnls = portfolio.analyzer.get_performance_stats_pnls()
stats_returns = portfolio.analyzer.get_performance_stats_returns()
stats_general = portfolio.analyzer.get_performance_stats_general()
```



i For detailed information about available statistics and creating custom metrics, see the [Portfolio guide](#). The Portfolio guide covers:

- Built-in statistics categories (PnLs, returns, positions, orders based).
- Creating custom statistics with `PortfolioStatistic`.
- Registering and using custom metrics.

Visualization

NautilusTrader provides interactive tearsheets and plots via Plotly:

```
from nautilus_trader.analysis import create_tearsheet

# After backtest run
engine.run()

# Generate interactive HTML tearsheet
create_tearsheet(engine, output_path="tearsheet.html")
```



This creates an interactive HTML report with:

- Equity curve
- Drawdown analysis

- Monthly returns heatmap
- Performance statistics table
- Returns distribution

For more control, generate individual plots:

```
from nautilus_trader.analysis import create_equity_curve

returns = engine.portfolio.analyzer.returns()
fig = create_equity_curve(returns, title="My Strategy Equity")
fig.show() # Display in browser
fig.write_image("equity.png") # Export to PNG (requires kaleido)
```



Install visualization dependencies:

```
uv pip install "nautilus_trader[visualization]"
```



Report generation patterns

Live trading

During live trading, generate reports periodically:

```
import pandas as pd

class ReportingActor(Actor):
    def on_start(self):
        # Schedule periodic reporting
        self.clock.set_timer(
            name="generate_reports",
            interval=pd.Timedelta(minutes=30),
            callback=self.generate_reports
        )

    def generate_reports(self, event):
        # Generate and log reports
        positions_report = self.trader.generate_positions_report()

        # Save or transmit report
        positions_report.to_csv(f"positions_{event.ts_event}.csv")
```



Performance analysis

For backtest analysis:

```
import pandas as pd

# Run the backtest
engine.run(start=start_time, end=end_time)

# Collect results
positions_closed = engine.cache.positions_closed()
stats_pnls = engine.portfolio.analyzer.get_performance_stats_pnls()
stats_returns = engine.portfolio.analyzer.get_performance_stats_returns()
stats_general = engine.portfolio.analyzer.get_performance_stats_general()

# Create summary dictionary
results = {
    "total_positions": len(positions_closed),
    "pnl_total": stats_pnls.get("PnL (total)"),
    "sharpe_ratio": stats_returns.get("Sharpe Ratio (252 days)"),
    "profit_factor": stats_general.get("Profit Factor"),
    "win_rate": stats_general.get("Win Rate"),
}

# Display results
results_df = pd.DataFrame([results])
print(results_df.T) # Transpose for vertical display
```



i Reports are generated from in-memory data structures. For large-scale analysis or long-running systems, consider persisting reports to a database for efficient querying. See the [Cache guide](#) for persistence options.

Integration with other components

The `ReportProvider` works with several system components:

- **Cache:** Source of all trading data (orders, positions, accounts) for reports.
- **Portfolio:** Uses reports for performance analysis and metrics calculation.
- **BacktestEngine:** Uses reports for post-run analysis and visualization.
- **Position snapshots:** Required for accurate PnL reporting in `NETTING` OMS.

Summary

The `ReportProvider` generates reports from orders, fills, positions, and account states as structured DataFrames for analysis and visualization. For accurate total PnL in `NETTING` OMS, include position snapshots when generating reports.

Related guides

- [Visualization](#) - Interactive tearsheets and charts from backtest results.
- [Portfolio](#) - Portfolio statistics and performance metrics.
- [Backtesting](#) - Running backtests that generate reports.
- [Cache](#) - Cache system that stores data for reports.

[< Portfolio](#)[Previous Page](#)[Logging >](#)[Next Page](#)